

A red arrow pointing to the right, positioned to the left of the title text.

Les Modèles - Django ORM

Les Modèles DJANGO

- **Présentation**

- Un modèle est la source d'information unique et définitive à propos des données.
- Il contient les champs et le comportement essentiels des données stockées.
- Généralement, chaque modèle correspond à une seule table de base de données.
- Chaque modèle est une classe Python qui hérite de `django.db.models.Model`.
- Chaque attribut du modèle représente un champ de base de données.
- Django offre une API d'accès à la base de données générée automatiquement.

Les Modèles DJANGO

- Modèles : exemple

```
# dans app/models.py
```

```
from django.db import models
```

```
class Article(models.Model):  
    texte = models.TextField()  
    date_de_publication = models.DateTimeField('date de publication')  
  
class Commentaire(models.Model):  
    article = models.ForeignKey(Article, on_delete=models.CASCADE)  
    commentaire = models.CharField(max_length=256)  
    date_de_commentaire = models.DateTimeField('date de commentaire')  
    plus_moins = models.IntegerField(default=0)
```

Les Modèles DJANGO

- Les champs
 - Les champs sont définis par des attributs de classe `models.xxxFields`
 - Django utilise les types des classes de champs pour déterminer un certain nombre de choses :
 - Le type du champ en base de données (par ex. INTEGER, VARCHAR, TEXT).
 - Le widget HTML par défaut à injecter dans un formulaire (par ex. input, select...).
 - Les exigences minimales de validation, utilisées dans l'administration de Django et dans les formulaires générés automatiquement.
 - La Structure des classes `models.xxxFields` est proche des classes `forms.xxxFields`
 - Il est possible de définir ses propres champs en héritant de la classe `django.db.models.Field`

<https://docs.djangoproject.com/fr/3.1/ref/models/fields/#model-field-types>

Les Modèles DJANGO

- Les paramètres de champs
- Chaque champ accepte un certain nombre de paramètres spécifiques.
- Ex : **CharField** exige le paramètre **max_length** à transmettre en paramètre du **VARCHAR SQL**
- D'autres paramètres sont communs à toutes les classes de champs :
 - **null** : stocke les valeurs vides avec NULL en base. False par défaut.
 - **blank** : le champ peut être vide à la validation du formulaire. False par défaut.
 - **choices** : Une séquence de tuples à 2 valeurs à utiliser comme liste de choix pour ce champ.
Fixe le widget à select au lieu de input.
Accès aux valeurs par **get_[nom_du_champ]_display**
 - **default** : La valeur par défaut du champ. peut être une valeur ou un objet exécutable
 - **help_text** : Texte d'aide supplémentaire à afficher avec le composant de formulaire.
 - **primary_key** : Le champ représentera la clé primaire du modèle.
 - **unique** : Le champ doit contenir des valeurs unique
 - **db_index** : un index sera créé sur ce champ par le moteur de la base
- Le premier paramètre positionnel est un nom verbeux du champ (commentaire)

Les Modèles DJANGO

- Options de meta
- On peut attribuer de façon facultative des métadonnées à un modèle en utilisant une classe Meta imbriquée :
- Les méta données concernent tout ce qui n'est pas les champs : options de tri (**ordering**), le nom de la table de base de données (**db_table**) ou des noms verbeux singulier et pluriel...

```
from django.db import models

class Ox(models.Model):
    horn_length = models.IntegerField()

    class Meta:
        ordering = ["horn_length"]
        verbose_name_plural = "oxen"
```

<https://docs.djangoproject.com/fr/3.1/ref/models/options/>

Les Modèles DJANGO

- **Modèle Abstrait**
 - On peut créer des modèles dont le but n'est pas de décrire une table en base de données, mais de servir de support d'héritage pour d'autres modèles
 - Pour se faire on utilise la meta **abstract = True**

```
from django.db import models
```

```
class CommonInfo(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.PositiveIntegerField()
```

```
class Meta:  
    abstract = True
```

```
class Student(CommonInfo):  
    home_group = models.CharField(max_length=5)
```

Les Modèles DJANGO

- Manipulation des modèles : création, enregistrement

```
from blog.models import Blog
```

```
# création d'un objet du modèle par le constructeur
```

```
b = Blog(name='Beatles Blog', tagline='All the latest Beatles news.')
```

```
# enregistrement en base. Save ne retourne aucune valeur
```

```
b.save()
```

```
# modification de l'objet en base
```

```
b.name = 'New name'
```

```
b.save()
```

```
# réactualisation du champ en base
```

```
del b.name
```

```
print(b.name)
```

```
>>> 'New name'
```


Les Modèles DJANGO

- **Manipulation des modèles : Sélection d'objets**
- Tout modèle possède un objet « **objects** » de classe **Manager** par défaut, qui permet de sélectionner des enregistrements de la table liée au modèle
- Les méthodes du Manager renvoient des objets **QuerySet** représentant des collections d'enregistrements, ou des objets uniques

```
all_entries = Entry.objects.all() # tous les enregistrements
```

```
# les enregistrements sélectionnés par les filtres sur les champs
```

```
collection = Entry.objects.filter(fieldx=value1, fieldy=value2)
```

```
# les enregistrements exclus par les filtres sur les champs
```

```
collection = Entry.objects.exclude(fieldx=value1, fieldy=value2)
```

```
# enchainements de filtre
```

```
collection = Entry.objects.filter(fieldx=value1).exclude(fieldy=value2)
```

```
object = Entry.objects.get(pk=1) # selection d'un objet unique
```

```
Entry.objects.filter(blog=b).delete() # # suppression
```

Les Modèles DJANGO

- Manipulation des modèles : sélection d'objets
- Les QuerySet sont différés ou « lazy » : les données en base ne sont collectées que lorsque le QuerySet est évalué (print, affectation, iteration, conversion...)

raccourci de recherche pour la primary key => pk

Blog.objects.get(pk=14) # quelque soit le nom du champ primary key

slicing

some_entries = Entry.objects.all()[:5]

tri

collection = Entry.objects.order_by(fieldx).desc()

recherche exacte (par_default)

object = Entry.objects.get(fieldx__exact='value1') # également __gt (>), __gte(<=), __lt (<)...

recherche exacte case insensitive

object = Entry.objects.get(fieldx__iexact='vAlUe1')

recherche d'une sous-chaine

object = Entry.objects.get(fieldx__contains='alu') # également __startswith et __endswith

Les Modèles DJANGO

- Manipulation des modèles : sélection d'objets

```
# recherche champ vide  
object = Entry.objects.filter(color__isnull=True)
```

```
# recherche dans un ensemble  
collection = Entry.objects.filter(id__in=[1, 3, 4])
```

```
# recherche dans un QuerySet  
inner_qs = Blog.objects.filter(name__contains='Cheddar')  
entries = Entry.objects.filter(blog__in=inner_qs)
```

```
# recherches par date python  
Entry.objects.filter(pub_date__date=datetime.date(2020, 1, 1))  
Entry.objects.filter(pub_date__date__gt=datetime.date(2020, 1, 1))  
Entry.objects.filter(pub_date__range=(start_date, end_date))  
Entry.objects.filter(pub_date__year=2020) # également __month, __day, __week...
```

```
# recherches par regex  
object = Entry.objects.get(title__regex=r'^(Une?|L[ea]) +')
```

Les Modèles DJANGO

- Manipulation des modèles : conversions de QuerySet

```
# Les QuerySet contiennent des objets Model par défaut
```

```
Blog.objects.filter(name__startswith='Beatles')
```

```
>>> <QuerySet [<Blog: Beatles Blog>]>
```

```
# avec values(), il contiennent des dictionnaires
```

```
Blog.objects.filter(name__startswith='Beatles').values()
```

```
>>> <QuerySet [{'id': 1, 'name': 'Beatles Blog', 'tagline': 'All the latest Beatles news.'}]>
```

```
# avec values_list(), il contiennent des tuples avec selection des champs
```

```
Entry.objects.values_list('id', 'headline')
```

```
>>> <QuerySet [(1, 'First entry'), ...]>
```

```
# avec date(), ils convertissent les champs date en dates python
```

```
Entry.objects.dates('pub_date', 'year')
```

```
>>> [datetime.date(2005, 1, 1)]
```

Les Modèles DJANGO

- Manipulation des modèles : Comparaison entre champs

```
# Entrées dont la date de modif est supérieur à 3j après la publication  
# F permet de transformer un champ en valeur de champs  
from datetime import timedelta  
from django.db.models import F  
Entry.objects.filter(mod_date__gt=F('pub_date') + timedelta(days=3))
```


Les Modèles DJANGO

- Manipulation des modèles : fonctions d'aggrégation

```
# COUNT.
```

```
Book.objects.count()
```

```
>>> 2452
```

```
# COUNT WHERE...
```

```
Book.objects.filter(publisher__name='BaloneyPress').count()
```

```
>>> 73
```

```
# AVG.
```

```
from django.db.models import Avg
```

```
Book.objects.all().aggregate(Avg('price'))
```

```
>>> {'price__avg': 34.35}
```

```
# MAX
```

```
from django.db.models import Max
```

```
Book.objects.all().aggregate(Max('price'))
```

```
>>> {'price__max': Decimal('81.20')}
```

Les Modèles DJANGO

- Manipulation des modèles : fonctions d'aggrégation

```
# Opérations sur des agrégats.
from django.db.models import FloatField
Book.objects.aggregate(
    price_diff=Max('price', output_field=FloatField()) - Avg('price'))
>>> {'price_diff': 46.85}

# GROUP BY
from django.db.models import Count
pubs = Publisher.objects.annotate(num_books=Count('book'))
pubs
>>> <QuerySet [<Publisher: BaloneyPress>, <Publisher: SalamiPress>, ...]>
pubs[0].num_books
>>> 73
```

Les Modèles DJANGO

- Les relations : « many to one – foreign key »
- Pour définir une relation **many-to-one**, entre modèles, on utilise **django.db.models.ForeignKey**.
- Il s'agit d'un attribut de classe d'un modèle, pareil aux autres classes `xxxField`.
- l'instance de `ForeignKey` prend en premier paramètre positionnel le modèle relié
- Le paramètre `on_delete` renseigne la politique à appliquer en cas de suppression de l'objet référencé

```
class Artist(models.Model):  
    name = models.CharField(max_length=10)  
  
class Album(models.Model):  
    artist = models.ForeignKey(Artist, on_delete=models.CASCADE)
```

Les Modèles DJANGO

- Les relations : « many to many »
 - Pour définir une relation **many-to-many**, entre modèles, on utilise `django.db.models.ManyToManyField`.
 - l'instance de `ManyToManyField` prend en premier paramètre positionnel le modèle relié
 - Par convention, la variable de ce type sera un mot au pluriel
 - La relation **many-to-many** n'est défini que dans un des deux modèles, celui qui correspond le mieux au formulaire

```
from django.db import models

class Topping(models.Model):
    pass

class Pizza(models.Model):
    toppings = models.ManyToManyField(Topping)
```

Les Modèles DJANGO

- « many to many » : paramètres supplémentaires
- Dans une relation **many-to-many** simple, une table de correspondance est créée automatiquement en base.
- Si l'on veut ajouter des champs supplémentaires dans la relation entre deux modèles, il faut utiliser un **modèle tiers** décrivant la relation
- On utilise alors le paramètre **through** de **ManyToManyField** pour spécifier ce modèle intermédiaire

```
class Person(models.Model):  
    name = models.CharField(max_length=128)  
  
class Group(models.Model):  
    name = models.CharField(max_length=128)  
    members = models.ManyToManyField(Person, through='Membership')  
  
class Membership(models.Model):  
    person = models.ForeignKey(Person, on_delete=models.CASCADE)  
    group = models.ForeignKey(Group, on_delete=models.CASCADE)  
    date_joined = models.DateField()  
    invite_reason = models.CharField(max_length=64)
```


Les Modèles DJANGO

- Les relations : « one to one »
- Pour définir une relation **one-to-one**, entre modèles, on utilise `django.db.models.OneToOneField`.
- L'instance de `OneToOneField` prend en premier paramètre positionnel le modèle relié
- Une relation **one-to-one** étend un modèle avec les propriétés d'un autre

```
class Place(models.Model):
    name = models.CharField(max_length=50)
    address = models.CharField(max_length=80)

class Restaurant(models.Model):
    place = models.OneToOneField(
        Place,
        on_delete=models.CASCADE,
        primary_key=True, # le champ OneToOne est la clé primaire de restaurant
    )
    serves_hot_dogs = models.BooleanField(default=False)
```

Les Modèles DJANGO

- Recherches sur les relations

```
# recherche sur le champ name du modèle Blog relié par le champ blog
Entry.objects.filter(blog__name='Beatles Blog')
```

```
# recherche sur un champ headline dans le sens inverse de la relation + opérateur contains
Blog.objects.filter(entry__headline__contains='Lennon')
```

```
# recherche à travers plusieurs relations Blog → Entry et Entry → Author
Blog.objects.filter(entry__authors__name='Lennon')
```

```
# GROUP BY HAVING sur une relation
#compte des livres ayant une note au dessus de 5 par éditeur
from django.db.models import Count, Q
above_5 = Count('book', filter=Q(book__rating__gt=5))
pubs = Publisher.objects.annotate(above_5=above_5)
pubs[0].above_5
>>> 23
```

Les Modèles DJANGO

- Mise en cache des QuerySet
 - l'évaluation d'un QuerySet déclenche la requête sur la base de donnée
 - Pour éviter de surcharger la base, il est fortement conseillé de mettre en cache les QuerySet dès que possible, et de travailler sur ce cache pour les traitements
 - La mise en cache n'est rien d'autre que l'affectation dans une variable
 - Il peut être utile de ne charger que certains champs d'une requête. On utilise `only()` ou `defer()` pour cela

```
# charger dans le QuerySet tout sauf age et biography  
Person.objects.defer("age", "biography")
```

```
# charger seulement name dans le QuerySet  
Person.objects.only("name")
```

Les Modèles DJANGO

- **ModelForm**
 - Un ModelForm est une classe utilitaire permettant de créer une classe de formulaire Form à partir d'un modèle Django.

```
from django.forms import ModelForm
from myapp.models import Article
```

```
class ArticleForm(ModelForm):
    class Meta:
        model = Article
        fields = ['pub_date', 'headline', 'content', 'reporter']
```

```
form = ArticleForm()
```

```
# Formulaire peuplé des données d'un enregistrement
article = Article.objects.get(pk=1)
form = ArticleForm(instance=article)
```

```
# Enregistre un objet en base de données et retourne le nouvel objet
f = ArticleForm(request.POST)
new_article = f.save()
```